Baku Apartment Prices from the Inside:
A From-Scratch Decision Tree and Pegasos SVM on bina.az Listings

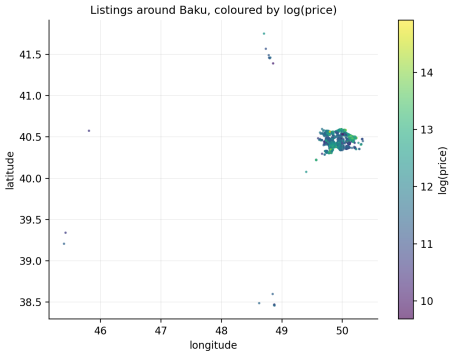
Hüseynova, T. — —

MLE-AI-201 Machine Learning ů AI Academy, Cohort II

October 2026

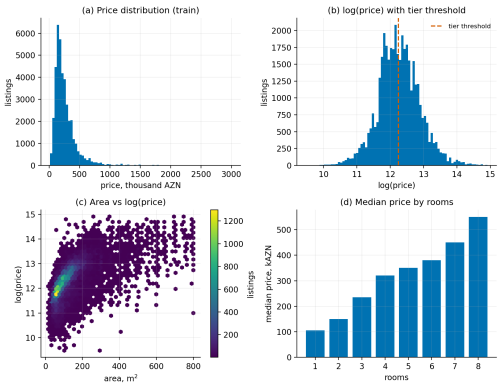
The task in one slide

- **Data:** 100,775 scraped bina.az sale listings → **57,421** clean residential rows, 55 features.
- **Task A (regression):** predict log(price) — our **decision tree**, MSE criterion.
- **Task B (classification):** premium vs. standard at the *train* median (208,000 AZN) — our **Pegasos SVM** + tree in classification mode.
- **Golden rule kept:** both models are our own NumPy; scikit-learn appears only as the baseline.



Cleaning a real dump (decisions, not magic)

- Keep *latest scrape* per listing: 100,775 → 64,454.
- Residential categories only (units differ elsewhere): → 58,452.
- Parse Azerbaijani strings: “145 m^2”, “7 / 9”, “1.3 sot” → numbers.
- Fixed rule-based outlier bounds (no quantiles ⇒ nothing leaks): → **57,421**.
- **Leakage dropped:** `unit_price`, `total_price`, identifiers, free text, views.
- Missing values: constant + explicit indicator column (no fitted statistics).



Split and framing

- Seeded 70/15/15 split: **40,195 / 8,613 / 8,613**.
- Tier threshold from the *training* median only; balance 49.9 / 50.5 / 48.9 % premium — balanced by construction.
- All tuning on validation; test touched once per final model.
- Metrics: RMSE/MAE/R^2 (log and AZN) ů precision/recall/F1, ROC-AUC, confusion matrix.
- One command reproduces everything: `python -m src.run_all` (seed 42).

Our decision tree (NumPy)

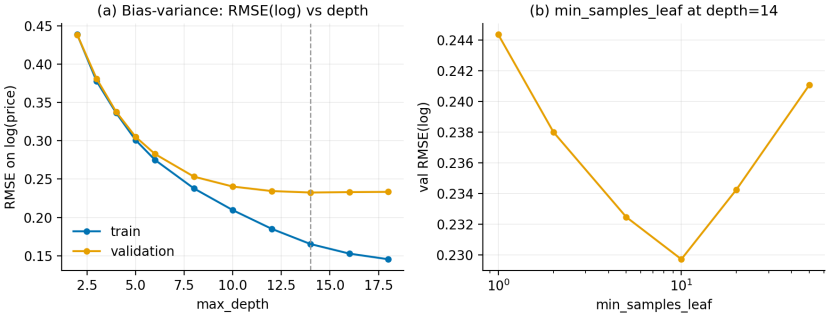
- CART-style recursion; **vectorised split search**: per feature sort once per node, score *all* thresholds via cumulative class counts / $\sum y, \sum y^2$.
- Criteria: Gini, entropy, MSE; sklearn-style weighted impurity decrease.
- Working stopping rules: `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease` (unit-tested).
- `predict_proba` from leaf frequencies; impurity-decrease feature importances.
- **Sanity test:** identical split (feature *and* threshold) to sklearn on fully-determined toy examples.

Our SVM: Pegasos, no QP solver

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_i \max\left(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right), \qquad \eta_t = \tfrac{1}{\lambda t}$$

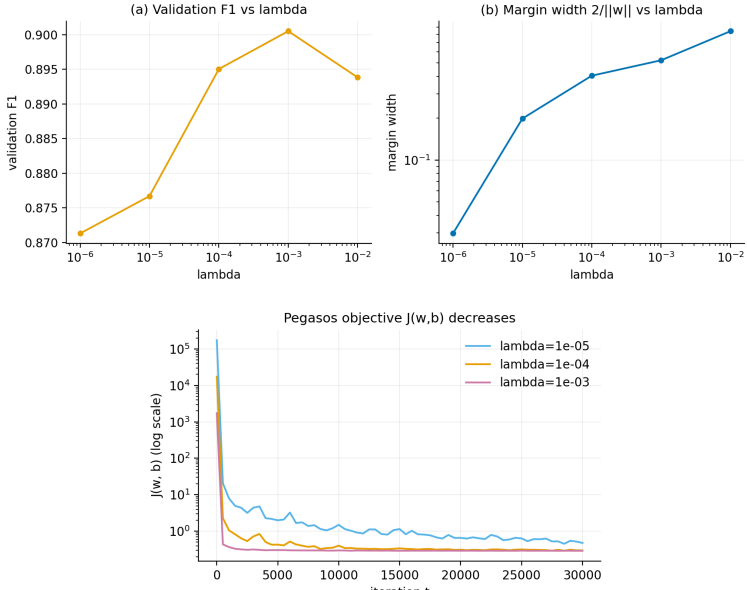
- Mini-batch (64) sub-gradient updates; bias unregularised; features standardised with train statistics.
- Margin you can see: $\|\mathbf{w}\| = 3.83$, width $2/\|\mathbf{w}\| = 0.52$; **11,693 / 40,195 (29%)** training points are support vectors.
- Non-linearity: **random Fourier features** (D=300) — keeps training $O(nD)$ and reuses the same solver (kernel Pegasos would be $O(n)$ per prediction at n=40k).

Tuning: the bias–variance story



- Train RMSE falls monotonically; validation bottoms at **depth 14** $\Rightarrow$ classic overfitting beyond.
- Gini vs. entropy: nearly identical (F1 0.9242 vs. 0.9225); Gini chosen on validation.

Tuning the SVM: margin vs. metric; convergence



Headline results (test set)

Task A regression on log(price)

	$\text{RMSE}_{\log}$	$R^2_{\log}$	MAE (AZN)
Ours: DecisionTree	**0.2311**	0.866	44,089
sklearn (matched)	0.2311	0.866	44,083

Task B price tier

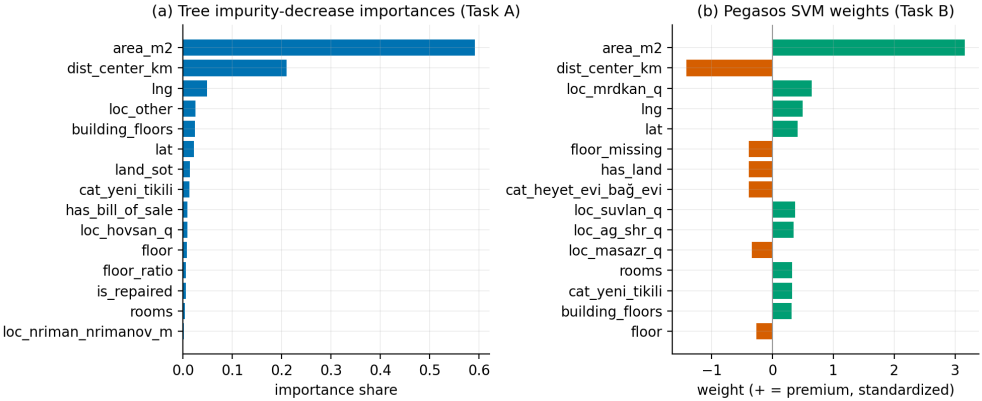
	F1	ROC-AUC
Ours: DecisionTree	**0.9200**	0.960
sklearn DecisionTree (matched)	0.9204	0.959
Ours: PegasosSVM (linear)	**0.8956**	0.953
sklearn SGDClassifier (hinge)	0.8960	0.953
Ours: Pegasos + RFF	0.8458	0.931
sklearn SVC-RBF (15k sub)	0.8383	0.923

5-fold CV: tree 0.2311 ± 0.0019 (= sklearn); Pegasos F1 0.899 ± 0.003 vs. SGD 0.900 ± 0.003.

Why the numbers agree — and where they don't

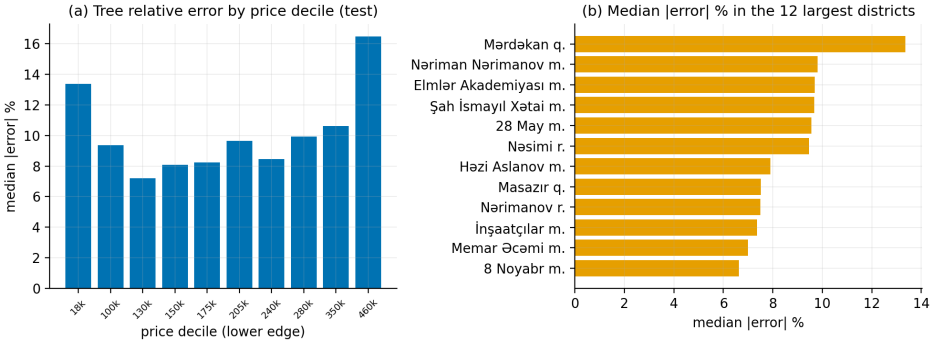
- Tree $\equiv$ sklearn to 3–4 decimals: same objective, same stopping rules; residual gap = **tie-breaking** (equal-gain splits, float midpoints), not statistics.
- Pegasos $\approx$ SGDClassifier(hinge): same loss, same schedule family.
- **Linear beats RBF here** (both ours and sklearn's): after one-hot encoding, 38/55 features are binary; the premium boundary is near-linear in area + centrality.
- Speed: our tree 4.7 s vs. sklearn 0.45 s (Cython, same $O(FN \log N)$); Pegasos 2.2 s; SVC-RBF needed a 15k subsample (16.6 s fit, 11.6 s predict).

What the models learned



- Both families agree: **area**, then geography (distance to centre, district), then floors/renovation.

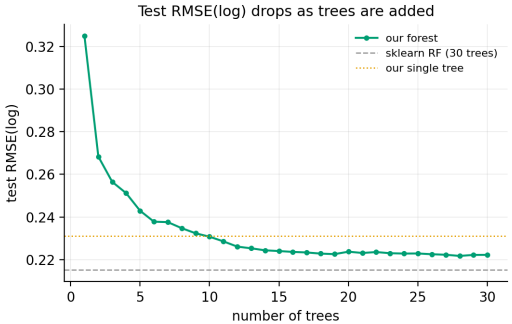
Error analysis: where and why they fail



- **60%** of tier misclassifications lie within $\pm 15\%$ of the 208k threshold — the label is nearly arbitrary there.
- Regression error doubles in the luxury decile and in peripheral / garden-house districts.

Bonuses

- **P8** Ridge via normal equations: $=$ sklearn to 10^{-15}; $\text{RMSE}_{\log}$ 0.303 — linear can't express area$\times$location.
- **P9** Forest of *our* trees (bootstrap $+$ per-split $\sqrt{d}$): $\text{RMSE}_{\log}$ 0.2311 $\rightarrow$ 0.2222, tracking sklearn's forest (0.2151).
- **P10** k-means segments $+$ PCA sharpen the error story: periphery is hardest.



Takeaways

- $\sim$250 lines of careful NumPy **reproduce scikit-learn exactly** on 57k real listings — the classical algorithms are small; the library's edge is engineering.
- Honest limits: asking prices (not transactions), Baku-only, unused text descriptions, threshold-band noise bounds Task B.
- Everything reproducible: seeded, pinned, `python -m src.run_all`, tag `v1.0-final`.

Thank you! Questions?